

JSON and Strings Exercise report: Aidan Khan and Jack McDonald

1. Importing the JSON file into an array using the Fetch API was no challenge.
2. Since the last time we have used the `map()` function was in CART 253 and for game items, it was a bit of a shock to no one that we could not remember exactly how to apply the `map()` function to arrays. We struggled with how the syntax worked in webAPI development, and reused randomiser code from previous projects to help pick the color from the `possibleColors` array,
3. As the `filter()` function only had to deal with one parameter within the JSON file, we considered cheating and removing the object from the file so as not to necessitate the use of computing power to parse one object out of a JSON file of nearly 200 items. Alas, it ended up being simpler to just filter the array.
4. Due to it's involvement with math functions within js, we struggled with the averaging function and `reduce()` for a good while. First, our structure was built in a way that the data from the JSON file could not be directly accessed as the "data" let was still inside the fetch block. The solution was to move the let *before* the fetch was called, and then assign the fetch to the variable, allowing the variable to persist across functions. Additionally, this task also got us familiar with the general structure of most array functions and how they are used in conjunction with JSON file arrays. Because JSONs have multiple parameters per index in their respective array, we struggled finding out how to access a specific parameter across all indexes, and it turned out to be just as simple as adding a `".(parameter)"` after the num specifier. Additionally, a component of the `reduce()` syntax we thought to be optional ended up being absolutely crucial to the working of this function: the `initialValue` parameter of 0, that goes after the function has been called outside of the brackets. This knowledge helped us greatly in breezing through the next few calculations.
5. Settling into a groove, steps 5-9 were very easy as we were able to apply previous syntax to each calculation, to the point where some could be copied and pasted with few parameters changed.
6. See above
7. See above
8. See above
9. See above
10. The `"toSorted()"` function originally sounded strange, as we knew how the `"sort()"` function worked and were perplexed as to why we would use the `"toSorted()"` version instead. However, when we discovered that `toSorted()` is for creating new arrays rather than sorting existing ones, it made sense, as we needed to maintain the original and sorted arrays for visualisation purposes.

As for our visualisation, we acknowledge that it is incredibly rudimentary. Applying knowledge from the previous task 6 and the garden task, we created a class for the iris that would later be pushed into an array with new parameters in a for loop, to create a field of divs that would represent our flowers. The challenge came in applying this previous knowledge and code in environments not suited specifically to their original counterparts. We sought to create 3 divs, one for the petal itself, one for the center of the petal (which would just be half the size of the original petal) and a sepal element that would lie at the bottom. However, in retrospect, given the inline nature of divs, we couldn't have hoped to construct this div flower as we hoped considering our methods. If we had used premade assets, we would have lost most of the benefit of using a class for irises, but we would have a much prettier looking field. Additionally, we wanted our interactive element to be two things: an ability to switch between the sorted and randomized fields at the press of a button, and a hover element showing each flower's numbers when the user moused over them. We got the data for the hover element to display, but floating divs that did not confine to the inline structure were impossible to create without disrupting the structure of the already existing field of irises. Additionally, we could not figure out how to clear and "re sort" the field of divs without just adding onto the existing one, which users could not even see because the flowers filled the screen. Chiefly among our problems is that no matter what we do, all flowers stay as one color upon load. The flowers do not correspond to their randomized colors, only the color of the last flower rendered. This seems like an easier fix and might actually be fixed by the time this is read.

If we were to do our task differently, we definitely could have started earlier and had more time to work out the kinks of our planned visualisation. Additionally, we could have figured out more ways to keep the user engaged, like animations and more interactivity. And finally, we could have made assets instead of using html elements to create a more beautiful visualisation. Half of the time on this project was spent in the console log wondering why some errors would not go away. This diminished our ability to focus on the most important part of the project. However, we were both caught up with many other projects at the time of development, so it is unfortunate we could not have devoted more time to this exercise.